

Algorithmic Design Techniques

Pre-Endsem Solutions

13 April 2026

Ten years ago you were so scared of such different things, but you survived them anyway. The same goes for five years ago and two years ago. Everything that has ever felt like a hurdle, you've passed through. So be afraid, identify your fears and then allow yourself to remember that in just a little while, this will be another thing that you have overcome.

— A Tumblr Screenshot

§1 Solutions

Problem 1.1.

Given a m , $\text{count_poly}(m)$ counts the number of polynomials P with coefficients from $\{0, 1, \dots, 9\}$ such that $P(3) = m$.

Input	Output
$m = 1$	1
$m = 3$	2
$m = 10$	4

Hint: Notice that there are no negative coefficients in P .

(a) Write pseudocode implementing $\text{count_poly}(m)$ in $O(m \log_3(m))$ time. **(50 Marks)**

(b) Argue the correctness of your implementation. **(30 Marks)**

The idea is that the degree of our polynomial is bounded by $d = \lfloor \log_3(m) \rfloor$ as otherwise $P(3) > m$.

We now make a DP table of size $(d + 1) \times m$ where $\text{dp}[i][j]$ is the number of polynomials with degree i -th such that $P(3) = j$.

We can use the transition relation

$$\text{dp}[i][j] = \sum_{\substack{k=0 \\ j-k \cdot 3^i \geq 0}}^9 \text{dp}[i-1][j - k \cdot 3^i]$$

And the initial values $\text{dp}[0][j] = \begin{cases} 1 & \text{if } 0 \leq j \leq 9 \\ 0 & \text{otherwise} \end{cases}$.

We can now work our way through i (top-down DP) and we would be done.

SOLUTION 1

```
1 def function count_poly(m):
```

 SOLUTION 1

```

2      d := ⌊log3(m)⌋
3      dp = 2D array of size (d + 1) × (m + 1), initialized to 0
4      for j ∈ range(0,10):
5          ⊥ dp[0][j] = 1
6      for i in range(1, d+1):
7          power = 3^i
8          for j in range(0, m+1):
9              for k in range(0,10):
10                 if j - k · power ≥ 0:
11                     ⊥ dp[i][j] = dp[i][j] + dp[i - 1][j - k · power]
12      return dp[d][m]
```

Notice, we fill a table of side $O(m \log_3(m))$ and every cell makes atmost $10 = O(1)$ calls to other values. Hence, the total time complexity is $O(m \log_3(m))$ as desired.

Problem 1.2. CWI has bought a new vending machine. Unfortunately, as CWI got a ‘bargain’ on it, the machine is also faulty. It has space for N items (1 indexed). However, when item i is dispensed; item $1, 2, \dots, i - 1$ are dispensed for free. If there are no snacks of some of the types $1, 2, \dots, i - 1$, simply no snack of this type is dispensed. Buying snack of type i is possible only if at least one snack of this type is available.

Prof. Wupendra Wulkarni wants to abuse the machine. The function `optimal_value(C, Q, k)` takes as input an (1-indexed) array C of length N where $C[i]$ is the cost of the i -th item, array Q of length N where $Q[i]$ is the quantity of the i -th item and k , the amount of money Prof. Wulkarni wishes to spend; and returns the maximum worth of items he could obtain.

For example, $k = 8$, $C = [7, 2, 3, 5, 7, 2]$ and $Q = [1, 3, 0, 3, 2, 1]$. We buy a snack of type 6; the vending machine also dispenses one snack of each of the types 1, 2, 4 and 5. We buy a snack of type 4; in addition to this snack, the vending machine dispenses one snack of type 2. So for a total value of 30.

(a) Write psudo-code implementing `optimal_value` in $O(Nk)$ (60 Marks)

(b) Argue the correctness of your implementation. (40 Marks)

This question is mainly bait for people who know too much algorithms. Given the framing, one could try using ideas similar to Bounded Knapsack. But, sadly, that would not really work.

The first observation you should make is that if the optimal solution is first buying i and then j , where $i < j$; then buying j first and then i cannot do worse.

The second observation worth making is if buying item i was a good idea; then buying it again (given everything from $1 \dots i$ is available) is still worth it.

And this allows us to use a greedy algorithm.

We simply calculate $B[i] = \frac{\sum_{j=1}^i C[j]}{C[i]}$ and buy item i maximizing $B[i]$. Notice, as qualities decrease and say it hits 0 for item j , the $B[i]$ for $i > j$ decreases. Hence, if $i < j$ and $B[i] > B[j]$; then that item is not worth buying.

All of these are $O(N)$ operations and we must make them atmost k times as k is decreasing. This gives us a time complexity of $O(Nk)$.

Translating this to pseudocode is left upto to the reader.

Remark: This question is a re-worded version of Problem A of 17th Polish Collegiate Programming Contest (2012). No solution to the question has been published anywhere. I took about 45 mins to solve it, but mainly as I wasted time thinking it had a Knapsack like approach.

Problem 1.3. Given two $n \times n$ matrices A and B , `magic_function(A,B)` allows us to compute their product AB in time $O(n^{\log_2 7}) \approx O(n^{2.81})$

(a) Assuming $m \geq n$, can you find an $O(m^2 n^{0.81})$ algorithm to multiply A , a $m \times n$ matrix, and B , a $n \times m$ matrix, using the above function. **Hint:** Multiply $\lceil \frac{m}{n} \rceil^2$ pairs of order- n matrices.

(40 Marks)

(b) Assuming $m \geq n \geq t$, Can you find an $O(mnt^{0.81})$ algorithm to multiply A , a $m \times n$ matrix, and B , a $n \times t$ matrix, using the above function. **Hint:** Multiply pairs of $t \times t$ matrices.

(40 Marks)

The hints are quite generous here.

For part (a), Since $m \geq n$, we can view the tall matrix A as a vertical stack of $n \times n$ blocks and the wide matrix B as a horizontal row of $n \times n$ blocks (upto padding with 0s). Let $q = \lceil \frac{m}{n} \rceil$.

Basically,

$$A = \begin{pmatrix} A_1 \\ A_2 \\ \dots \\ A_q \end{pmatrix}, B = (B_1 \ B_2 \ \dots \ B_q)$$

where each A_i and B_j is an $n \times n$ block. The resulting product $C = AB$ is an mm matrix consisting of q^2 blocks, where each block C_{ij} is defined by:

$$C_{ij} = A_i \cdot B_j$$

Since each A_i and B_j is $n \times n$, we can compute each C_{ij} using the `magic_function` in $O(n^{2.81})$.

We perform q^2 multiplications. Since $q \approx \frac{m}{n}$, the total time is:

$$O\left(\left(\frac{m}{n}\right)^2 n^{2.81}\right) = O(m^2 n^{0.81})$$

For part (b), Now we have $m \geq n \geq t$. The goal is to reach $O(mnt^{0.81})$. Following the logic from Part (a), we should partition everything into $t \times t$ blocks (upto padding by 0s).

1. Partition A ($m \times n$) into a grid of blocks $A_{i,j}$ where $1 \leq i \leq \frac{m}{t}$ and $1 \leq j \leq \frac{n}{t}$.
2. Partition B ($n \times t$) into a vertical stack of blocks $B_{j,1}$ where $1 \leq j \leq \frac{n}{t}$.
3. The resulting matrix C ($m \times t$) will have $\frac{m}{t}$ blocks.

Each block $C_{i,1}$ of the output is computed as:

$$C_{i,1} = \sum_{j=1}^{\frac{n}{t}} A_{i,j} \cdot B_{j,1}$$

To compute one block $C_{\{i,1\}}$, we perform $\frac{n}{t}$ multiplications of $t \times t$ matrices. There are $\frac{m}{t}$ such blocks in C . Hence, we made a total of $\frac{m}{t} \cdot \frac{n}{t} = \frac{mn}{t^2}$ multiplications. Each multiplication takes $O(t^{2.81})$.

Hence, the total time complexity is $O(mnt^{0.81})$.

Problem 1.4. Kitty and Min-Ho are trying to plan the perfect date across Seoul.

Let Seoul be a $m \times n$ grid with all important sites on the intersections. The cuteness of a site at the intersection of line i and line j is $C[i][j]$.

It takes $|i_1 - i_2| + |j_1 - j_2|$ to travel from (i_1, j_1) to (i_2, j_2) . Visiting a site takes unit time.

As Kitty and Minho love each others company, they gain one cuteness for every unit time they spend traveling.

Also, things should get more legendary as the date progresses. So they cuteness of a site visited later can't be less than one visited earlier.

Kitty and Minho have some t units of time before Kitty's flight back home. They start at $(0, 0)$, the location of KISS or Korean Independent School of Seoul, which provides no cuteness.

(a) Write the complete pseudocode for a recursive algorithm $\text{max_cute_tour}(C, t)$ that given the cutness table, returns the maximum cuteness a tour can obtain in time t . **(40 Marks)**

(b) In terms of m, n, t , What is the runtime of your recursive solution? **(20 Marks)**

(c) Write the pseudocode for the memoized version of your algorithm from part (a) without using any libraries or decorators or language features. **(30 Marks)**

(d) Modify your pseudocode from part (c) and now output the tour achieving maximum cuteness. **(30 Marks)**

This is just a classic DP problem. We can define a recurrence relation on current site and time remaining.

$$f(x, y, z) = \max_{\substack{C[i][j] > C[x][y] \\ z \geq |i-x| + |j-y| + 1}} f(i, z - |i-x| - |j-y| - 1) + C[i] + |i-x| + |j-y|$$

And our answer is $f(0, 0, t)$.

Doing this recursively has time complexity $O(mn^{\min(mn, t)})$.

Memoizing this is standard exercise and reduces the time complexity $O(mnt)$.

Finally, we can output the tour by converting to a dp and storing an additional pointer to which cell to visit next in the table.

Note: This is perhaps the most standard question. The point is to check if you have understood all the concepts taught in the course and not ask for any clever observations.

Remark: I have been told that we can do better as well. We can consider the locations in order of cuteness. At the cost of sorting, we can do some optimizations. Most of these are non-trivial and are based on weird tricks like Quadrangle Inequality and Aliens DP and Slope Trick. I am not well versed in these.

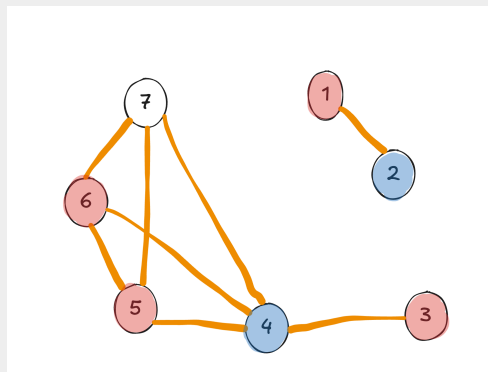
Problem 1.5. Master Tengen has sensed a massive surge in cursed energy across the n districts of Japan. To maintain the peace, the two rival schools: Tokyo Jujutsu High and Kyoto Jujutsu High, have both requested permission to station sorcerers across the country.

There are n districts in Japan, some of which are connected by high-speed transit routes. To ensure every citizen is safe from Curses, each school (Tokyo and Kyoto) insists that for every single district:

- The district must have a Sorcerer Station from their school, **OR**
- The district must be directly connected to another district that has one.

Master Tengen, however, is wary of the intense rivalry between the two schools. He fears that if a single district hosts stations from both Tokyo and Kyoto, the clashing Cursed Energy will resonate, accidentally birthing a Special Grade Disaster Curse.

See an example below. Red is Tokyo and Blue is Kyoto.



(a) As a practitioner of the Algorithms Jutsu, you are requested to provide a pseudo-code implementing a function `deployment_plan(G)` that takes the graph with vertices representing the cities of Japan and edges representing connection by a high-speed transit; provides a deployment plan (an array D such that $D[i]$ is 0 if no one is deployed there, 1 if Tokyo High is deployed there and 2 if Kyoto high is deployed there) for the sorcerers of the two schools if possible or returns NO if it is not possible. **(70 Marks)**

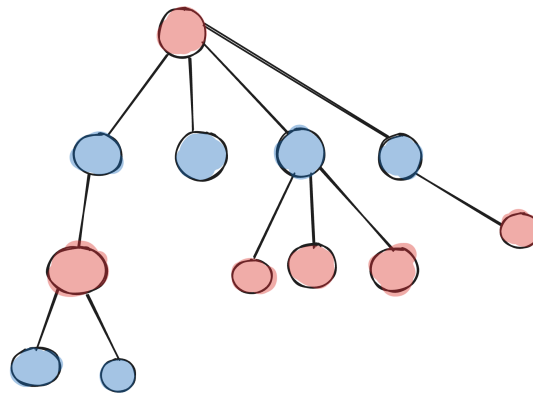
(b) Argue that your algorithm runs in $O(|V| + |E|) = O(n + \binom{n}{2}) = O(n^2)$ time. **(25 Marks)**

(c) Argue the correctness of your algorithm. **(25 Marks)**

The question doesn't ask for the smallest number of sorcerers to deploy. So why even leave any city empty?

Second, notice that we can clearly deal with the components of the graph separately.

Finally, in a connected component, we can always find a tree (remember, a tree is the minimally connected graph. So any other connected graph must have a tree embedded). And just filling a tree with Tokyo and Kyoto sorcerers at alternate depths fulfills the criteria.



This tells us the only NO instance is if there is some lone vertex. This takes $O(|V|)$ time to check.

Now one could separate the components and then find the tree and then do this but that is all too complex. Now that we know a solution exists, we can be sloppy/smart.

Answer 1: Choose an edge. If both the vertices are not colored, color them in opposite colors (note, if one of the vertex is colored, we can just color the other one in the opposite color). Keep doing till there is no uncolored vertex left.

Answer 2: Color some random vertex red. Color all its neighbors blue. Color the neighbors of all the blue vertices (those who are not already colored) red and so on. Repeat till no vertex is left uncolored.

Remark: The tree we obtain from A1 is similar to something we get via Kruskal's algorithm while the one in A2 is similar to the order of exploration in a simple BFS. As both of these lead to trees where the depths alternate in color, we are done by our previous argument.

Note: This problem is based on a problem titled "Guilds" from the 17th Polish Olympiad in Informatics (2010). I got the problem via the excellent book "Looking For a Challenge".